TOPIC:- **Educational Resource Sharing Platform**

## 1. Scenario:

The **Educational Resource Sharing** project aims to provide a secure, scalable, and high-performing platform for sharing educational materials. Leveraging AWS cloud infrastructure, this solution enables educators and students to upload, manage, and retrieve resources efficiently while ensuring security and reliability. The architecture supports both public-facing web servers for user interaction and private back-end databases for secure data storage.

## 2. Problem Statement:

Educational platforms often struggle with securely storing and sharing materials while managing high user demand. They face challenges like data breaches, poor scalability, and inconsistent performance. This project aims to resolve these issues by creating a secure, scalable, and reliable resource-sharing platform using AWS infrastructure.

## 3. Objectives:

1. Build a cloud-based platform for sharing educational files securely.
2. To integrate a scalable backend database (Amazon RDS) with secure connectivity..
3. Configure scalable storage and efficient file distribution using S3 and CloudFront.
4. Set up a VPC with public and private subnets for secure resource segregation.
5. Integrate Lambda for automation of backend operations like file processing.

## 4. Outcomes:

1. A functional platform for secure resource management and sharing.
2. Improved scalability and reliability, even under high user demand.
3. Enhanced security with private subnet deployment for sensitive data storage.
4. Efficient file delivery via S3 and CloudFront for faster user access.

## 1. Proposed AWS Components:

- **Amazon EC2**: For hosting the application.
- **Amazon RDS (Relational Database Service)**: For storing structured data.
- **Amazon S3**: For hosting static content like images and videos.
- **Amazon CloudFront**: As a Content Delivery Network (CDN) for faster delivery of assets.
- **AWS IAM**: To manage user permissions.
- **Amazon CloudWatch**: For monitoring performance and resource utilization.
- **AWS Lambda**: Automates backend tasks like file processing and event handling.
- **NAT Gateway**: Enables private subnet resources to access the internet securely.
- **Security Groups**: Manage inbound and outbound traffic for EC2 and RDS instances.

## 2. Solution:

- **Network Architecture**:

- A VPC with public subnets for EC2-hosted web servers and private subnets for RDS database ensures secure resource segregation and communication.
- NAT Gateway enables internet access for private resources securely.

- **Web Application**:
  - Hosted on EC2 in a public subnet for file uploads and database interaction.
  - Uses IAM roles to securely access S3 for file storage and retrieval.

- **Database**:
  - Amazon RDS is configured in a Single-AZ setup for development and testing. Security groups ensure restricted access to database resources.

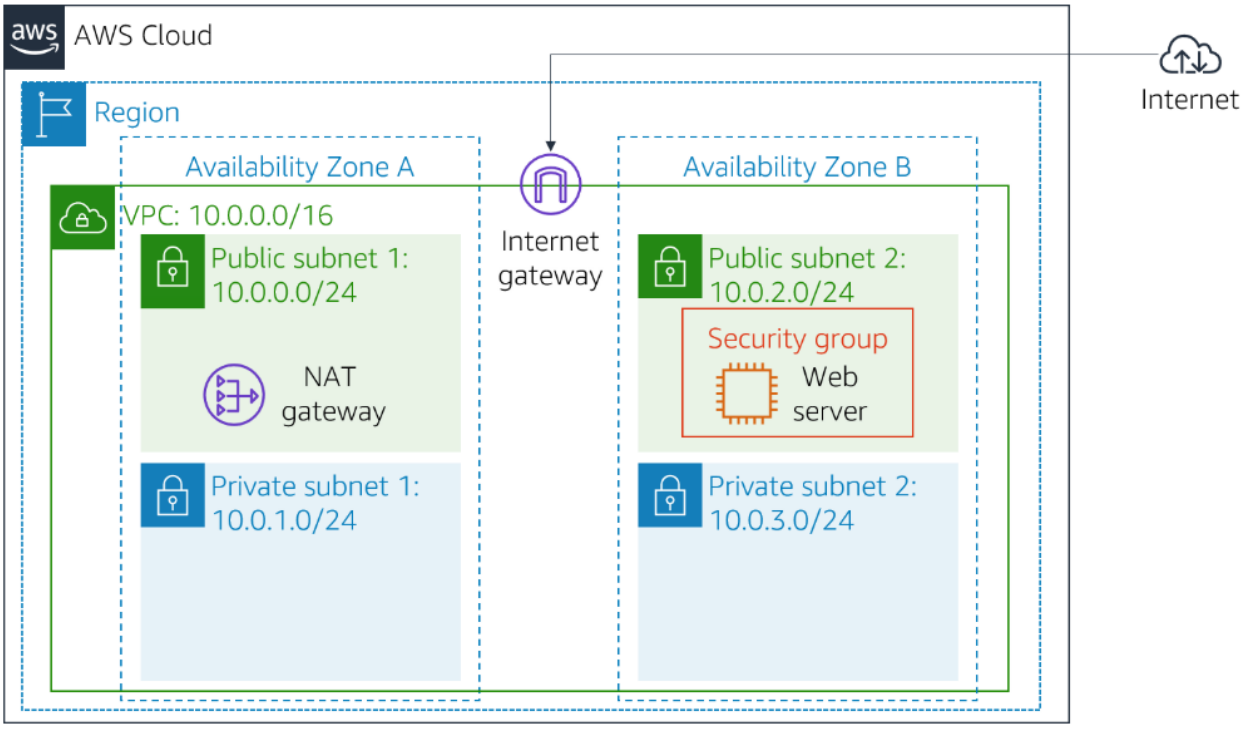
- **File Storage**:
  - Files are stored in S3, with CloudFront ensuring fast and secure delivery to users globally.

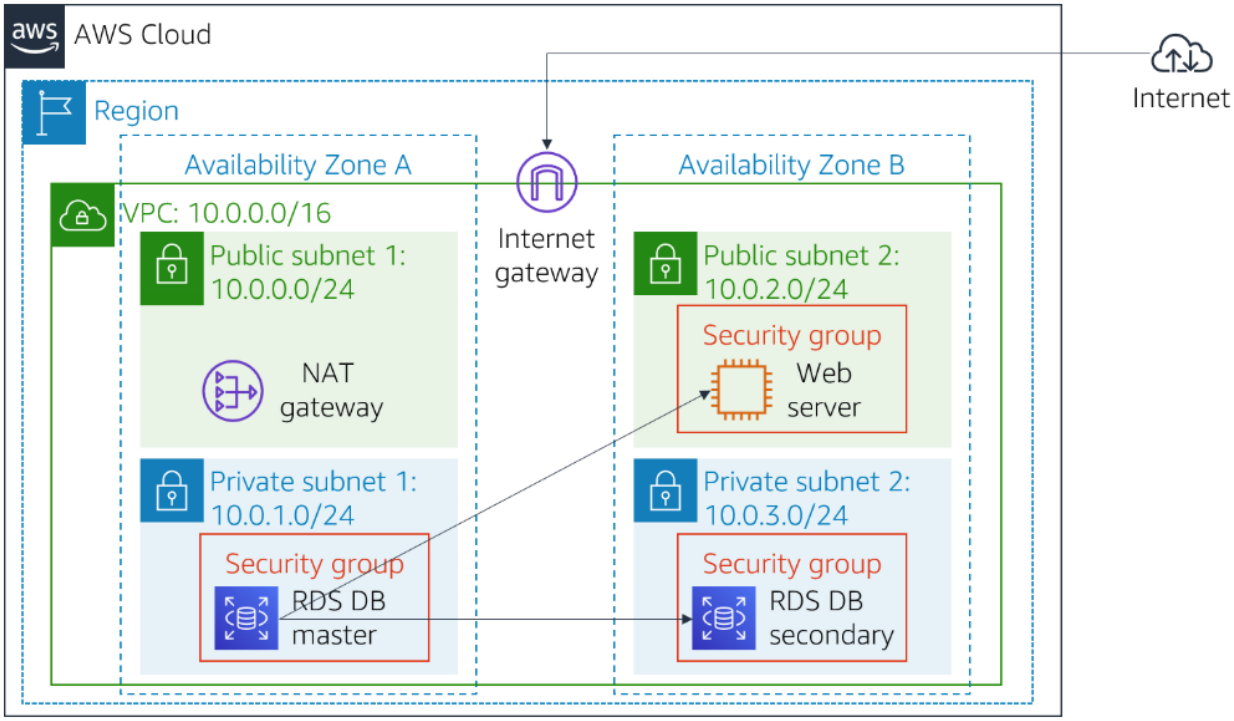
- **Automation**:
  - AWS Lambda processes files automatically after upload, optimizing backend operations and event handling.

## STEPS AND SCREENSHOTS

1.create a VPC:-

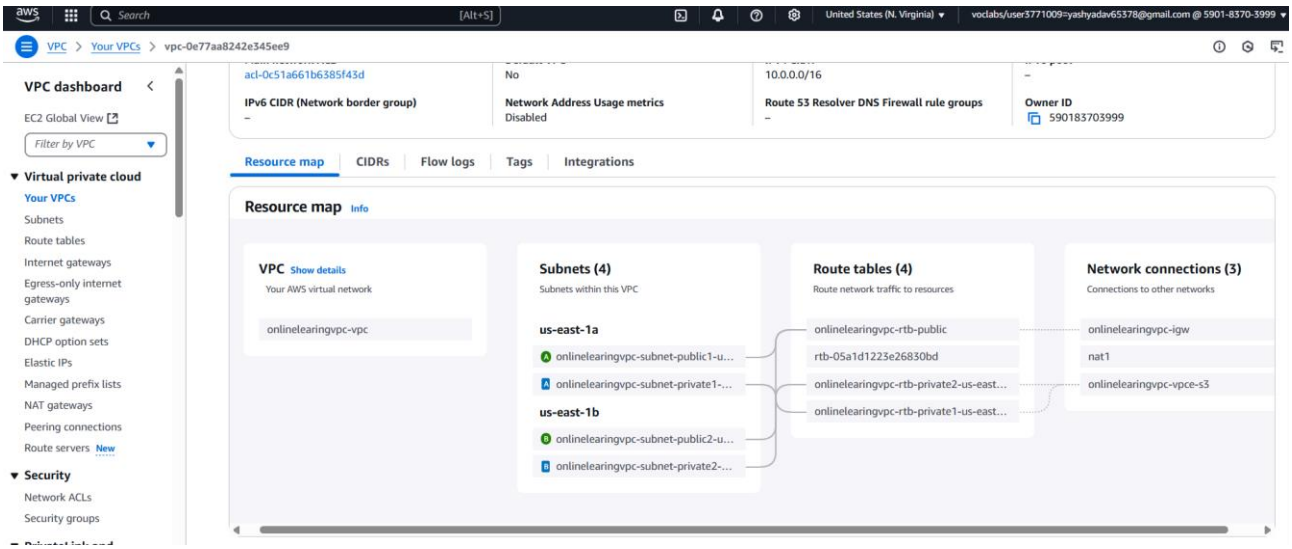
Keeping in mind:-



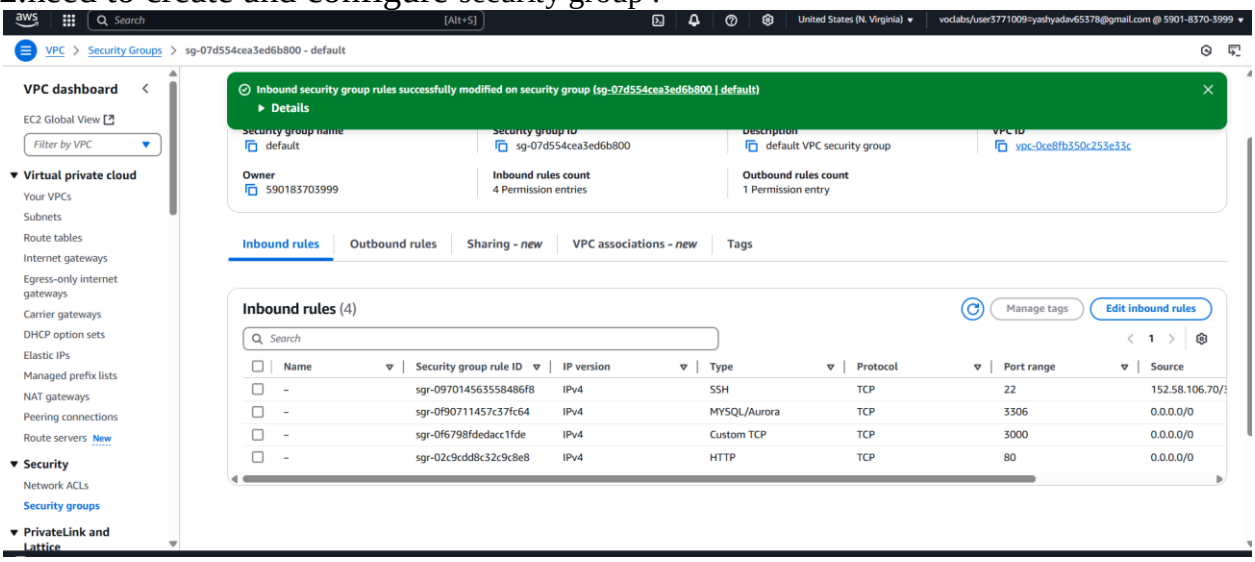




*Due to sandbox restrictions I am only able to create RDS database using Freetier.

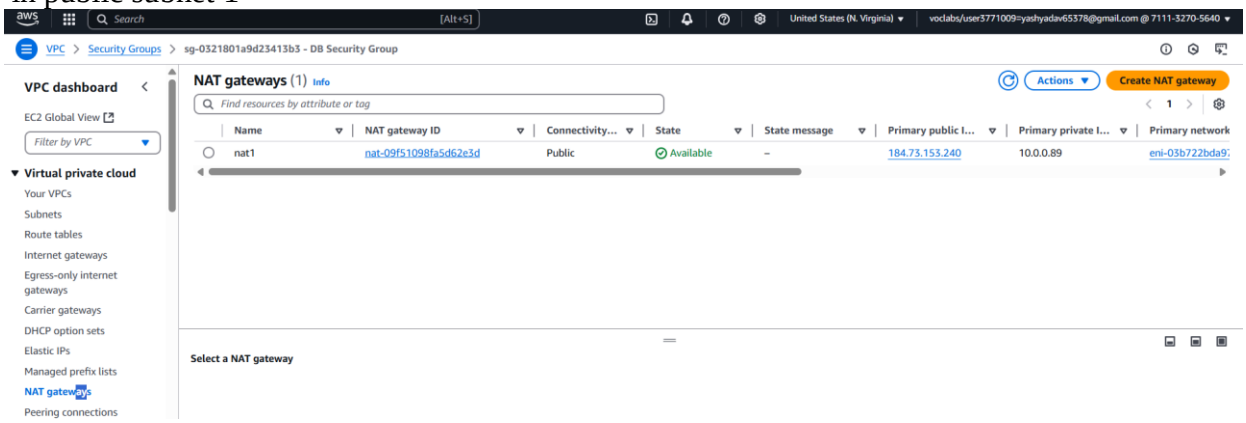
But for VPC and other thing where are not restrictions I am using the above Figure for My scenario.



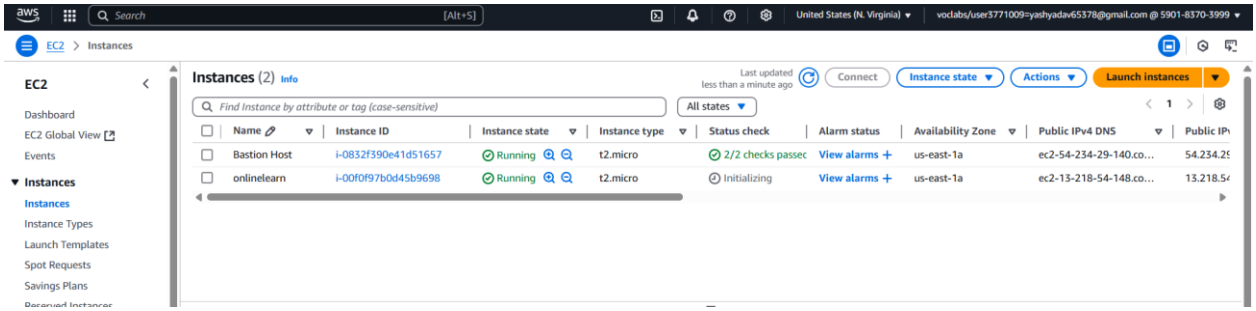

2.need to create and configure security group :-




3. Creating NAT getway:-
   -in public subnet 1



4. Creating EC2 instance:-



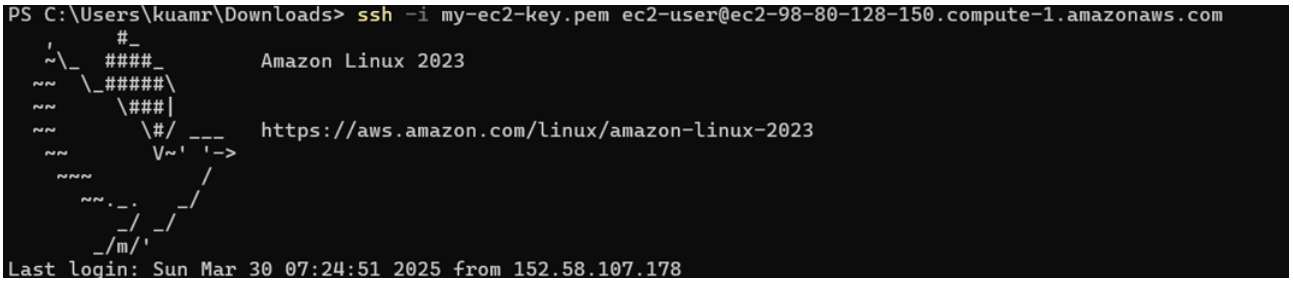
**It works!**

In Add user add this for the installation and setup of an Apache HTTP server:-

```
#!/bin/bash
yum update -y
yum install httpd -y
systemctl start httpd
systemctl enable httpd
```

5.After creating ec2 Open Poweshell and log in Ec2 though CLI:-

```
ssh -i <key.pem> ec2-user@<DNS>
```




- check if the its is working or not using this:-

```
curl http://localhost
= <html><body><h1>It works!</h1></body></html>
```

-check for updates:-

```
sudo yum update –y
```

6.create a folder and file for backend in EC2 using these commands :-

```
mkdir onlinelearning
cd onlinelearning
touch index.js
```

7. install node.js in EC2:-

```
curl -fsSL https://rpm.nodesource.com/setup_16.x | sudo bash -
sudo yum install -y nodejs
```

also check for version:-

```
node -v
npm -v
```

```
       _/ _/
     _/m/'
ast login: Sun Mar 30 07:32:49 2025 from 152.58.107.178
ec2-user@ip-172-31-16-234 ~]$ curl -fsSL https://rpm.nodesource.com/setup_18.x | sudo bash -
udo yum install -y nodejs
025-03-30 07:54:42 - Cleaning up old repositories...
025-03-30 07:54:42 - Old repositories removed
025-03-30 07:54:42 - Supported architecture: x86_64
025-03-30 07:54:42 - Added N|Solid repository for LTS version: 18.x
025-03-30 07:54:42 - dnf available, updating...
ode.js Packages for Linux RPM based distros - x86_64                                    8.7 MB/s | 1.0 MB     00:00
etadata cache created.
|Solid Packages for Linux RPM based distros - x86_64                                    7.0 MB/s | 637 kB     00:00
etadata cache created.
025-03-30 07:54:43 - Repository is configured and updated.
025-03-30 07:54:43 - You can use N|solid Runtime as a node.js alternative
025-03-30 07:54:43 - To install N|solid Runtime, run: dnf install nsolid -y
025-03-30 07:54:43 - Run 'dnf install nodejs -y' to complete the installation.
ast metadata expiration check: 0:00:01 ago on Sun Mar 30 07:54:43 2025.
ependencies resolved.
================================================================================================================================
Package                     Architecture                Version                                 Repository                  Size
================================================================================================================================
nstalling:
nodejs                      x86_64                      2:18.20.8-1nodesource                   nodesource-nodejs           34 M

ransaction Summary
================================================================================================================================
nstall  1 Package

otal download size: 34 M
nstalled size: 98 M
ownloading Packages:
odejs-18.20.8-1nodesource.x86_64.rpm                                                     36 MB/s |  34 MB     00:00
--------------------------------------------------------------------------------------------------------------------------------
otal                                                                                     36 MB/s |  34 MB     00:00
ode.js Packages for Linux RPM based distros - x86_64                                    106 kB/s | 3.1 kB     00:00
mporting GPG key 0x3AF28A14:
Userid     : "Nodesource Operations <operations@nodesource.com>"
Fingerprint: 242B 8138 31AF 0956 2B6C 46F7 6B88 DA4E 3AF2 8A14
```

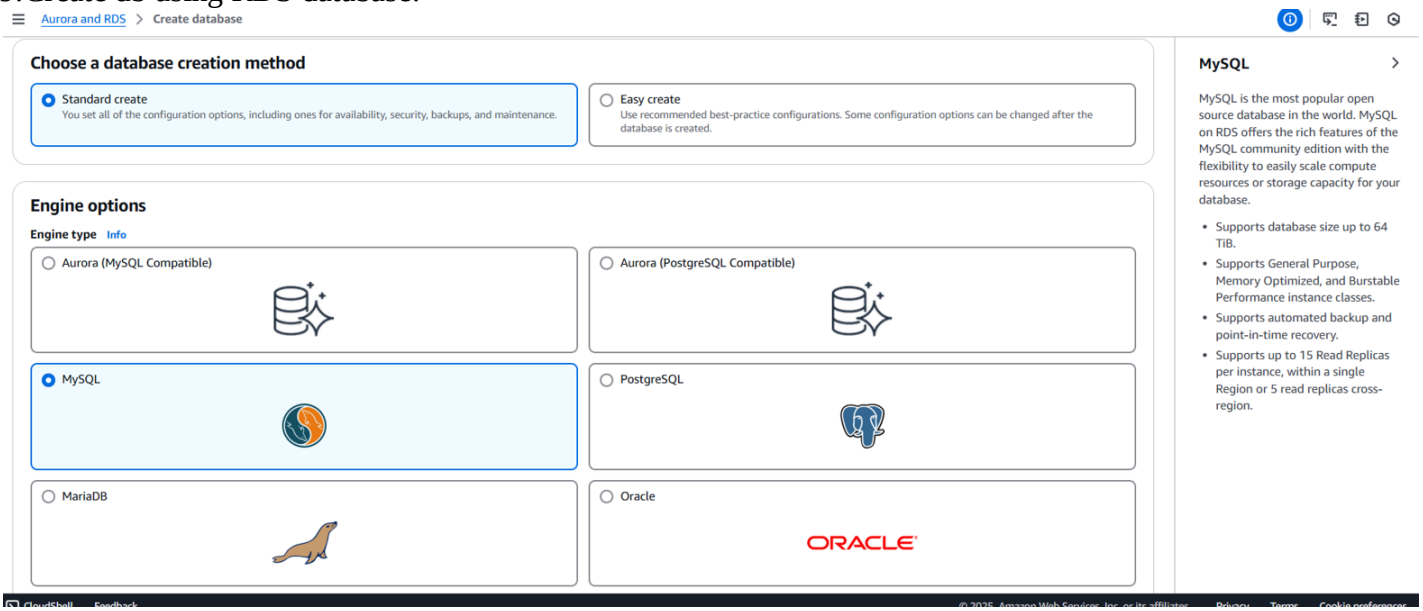
8. open file and put some thing in it to check :-

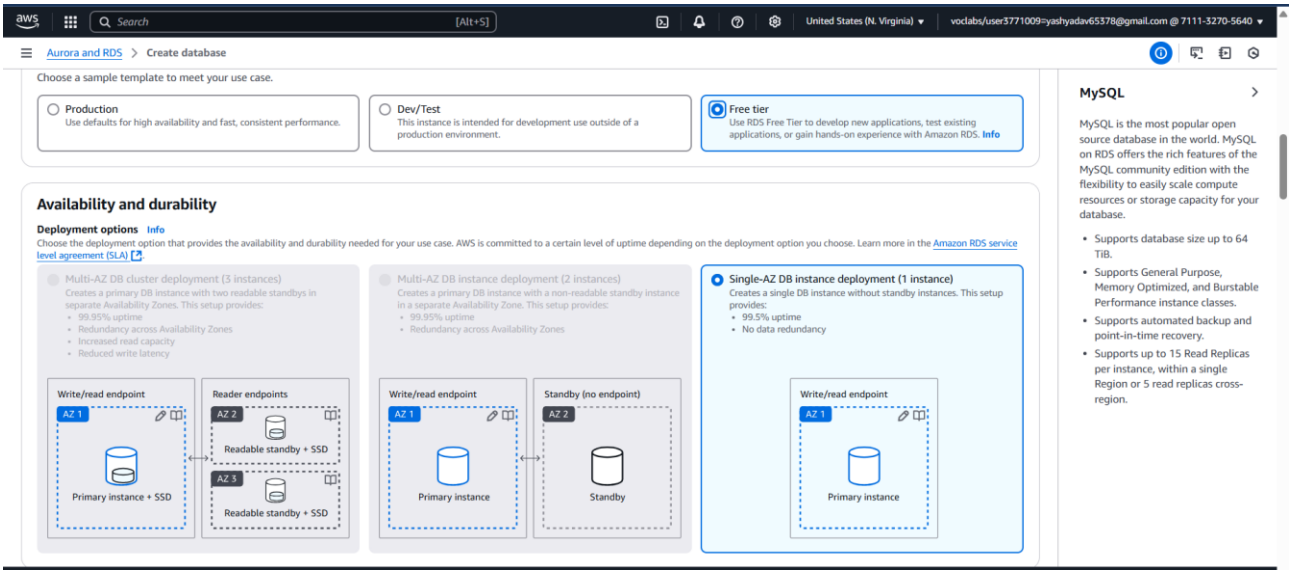
```
nano index.js
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Welcome to the Online Learning Platform API!');
});

app.listen(port, '0.0.0.0', () => {
  console.log(`Server is running on port ${port}`);
});
```

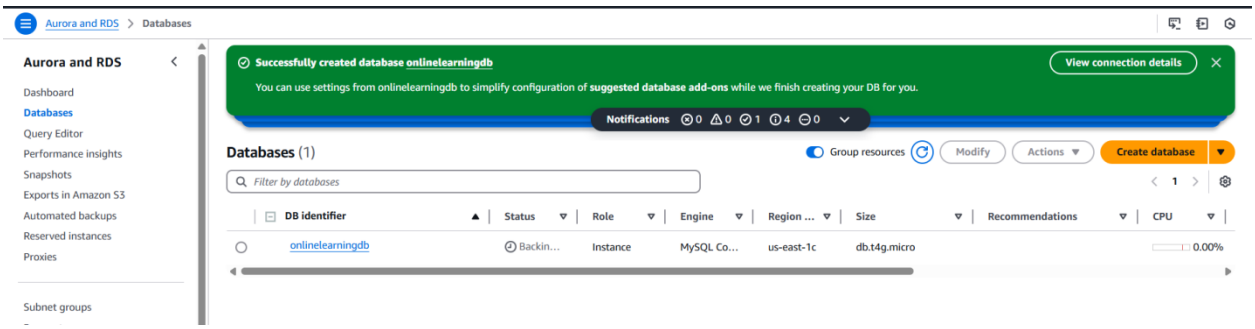
Check from browser:-

Welcome to the Online Learning Platform API!

9.Create db using RDS database:-






Come back to cli and install mysql:-

```
sudo dnf install mariadb105
mysql -h < Endpoint > -u admin –p
```

ex:- mysql -h onlinelearningdb.chyic2qaa8mv.us-east-1.rds.amazonaws.com -u admin -p

password:- enter the admin's password

Create Tables:-

```
CREATE DATABASE onlinelearning;
USE onlinelearning;

CREATE TABLE courses (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    instructor VARCHAR(255) NOT NULL
);

CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    course VARCHAR(255) NOT NULL
);
```

Insert Sample Data:-

```
INSERT INTO courses (name, instructor) VALUES
('Math 101', 'Dr. Sharma'),
('Science Basics', 'Dr. Verma'),
('History of Art', 'Dr. Kapoor');

INSERT INTO students (name, course) VALUES
('Aman Singh', 'Math 101'),
('Priya Sharma', 'Science Basics'),
('Rahul Gupta', 'History of Art');
```

**-Install MySQL Driver in EC2**:

-install the MySQL client library for Node.js:

```
npm install mysql2
```

-also update the code in index.js(our backend file):-

```
const express = require('express');
const mysql = require('mysql2');
```

```
const app = express();
const port = 3000;

// Establish connection to the MySQL database
const connection = mysql.createConnection({
  host: 'onlinelearningdb.chyic2qaa8mv.us-east-1.rds.amazonaws.com', // Replace with your RDS
endpoint
  user: 'admin', // Replace with your DB username
  password: '<your-password>', // Replace with your DB password
  database: 'onlinelearning' // Name of the database
});

// Connect to the database and handle errors
connection.connect((err) => {
  if (err) {
    console.error('Error connecting to the database:', err);
    process.exit(1); // Exit the process if unable to connect
  } else {
    console.log('Connected to the MySQL database!');
  }
});

// Root endpoint
app.get('/', (req, res) => {
  res.send('Welcome to the Online Learning Platform API!');
});

// Endpoint to get courses
app.get('/courses', (req, res) => {
  const instructor = req.query.instructor; // Extract 'instructor' from query parameters
  let query = 'SELECT * FROM courses';
  const queryParams = [];

  // If an instructor is provided, filter courses
  if (instructor) {
    query += ' WHERE instructor = ?';
    queryParams.push(instructor);
  }

  connection.query(query, queryParams, (err, results) => {
    if (err) {
      console.error('Error fetching courses:', err);
      res.status(500).send({ error: 'Something went wrong while fetching courses!' });
    } else {
      res.json(results);
    }
  });
});

// Endpoint to get students
app.get('/students', (req, res) => {
  connection.query('SELECT * FROM students', (err, results) => {
```

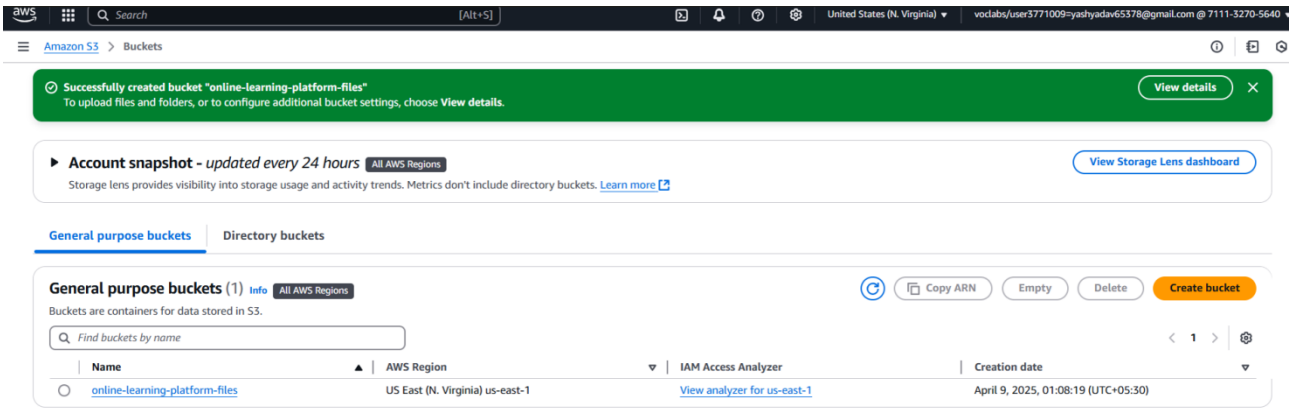
```
    if (err) {
      console.error('Error fetching students:', err);
      res.status(500).send({ error: 'Something went wrong while fetching students!' });
    } else {
      res.json(results);
    }
  });
});

// Start the server
app.listen(port, '0.0.0.0', () => {
  console.log(`Server is running on port ${port}`);
});
```

Then test:-

```
[ec2-user@ip-172-31-94-214 ~]$ curl http://3.95.169.185:3000/
Welcome to the Online Learning Platform API![ec2-user@ip-172-31-94-214 ~]$
```

```
[ec2-user@ip-172-31-94-214 ~]$ curl "http://3.95.169.185:3000/courses?instructor=Dr.%20Sharma"
[{"id":1,"name":"Math 101","instructor":"Dr. Sharma"}][ec2-user@ip-172-31-94-214 ~]$
```

10.Creating S3 Bucket for storing recourses :-




-Add Bucket polices:-

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": "*",
            "Action": [
                "s3:PutObject",
                "s3:GetObject",
                "s3:ListBucket"
            ],
            "Resource": [
                "arn:aws:s3:::online-learning-platform-files",
                "arn:aws:s3:::online-learning-platform-files/*"
```

```
            ]
        }
    ]
}
```

-Installing  sdk and multer:-

```
npm install aws-sdk
npm install multer
```

update the code in backend file S3:-

```
const express = require('express');
const mysql = require('mysql2');
const AWS = require('aws-sdk');
const multer = require('multer'); // For handling file uploads

const app = express();
const port = 3000;

// Set up AWS S3 configuration
AWS.config.update({
    region: 'us-east-1', // Replace with your bucket region
});
const s3 = new AWS.S3();

// Configure Multer to store files in memory for easy upload
const upload = multer({ storage: multer.memoryStorage() });

// Establish connection to the MySQL database
const connection = mysql.createConnection({
    host: 'onlinelearningdb.chyic2qaa8mv.us-east-1.rds.amazonaws.com', // Replace with your RDS
endpoint
    user: 'admin', // Replace with your DB username
    password: 'Yash990yash', // Replace with your DB password
    database: 'onlinelearning' // Name of the database
});

// Connect to the database and handle errors
connection.connect((err) => {
    if (err) {
        console.error('Error connecting to the database:', err);
        process.exit(1); // Exit the process if unable to connect
    } else {
        console.log('Connected to the MySQL database!');
    }
});

// Root endpoint
app.get('/', (req, res) => {
    res.send('Welcome to the Online Learning Platform API!');
});

// Endpoint to get courses
```

```
app.get('/courses', (req, res) => {
    const instructor = req.query.instructor; // Extract 'instructor' from query parameters
    let query = 'SELECT * FROM courses';
    const queryParams = [];

    // If an instructor is provided, filter courses
    if (instructor) {
        query += ' WHERE instructor = ?';
        queryParams.push(instructor);
    }

    connection.query(query, queryParams, (err, results) => {
        if (err) {
            console.error('Error fetching courses:', err);
            res.status(500).send({ error: 'Something went wrong while fetching courses!' });
        } else {
            res.json(results);
        }
    });
});

// Endpoint to get students
app.get('/students', (req, res) => {
    connection.query('SELECT * FROM students', (err, results) => {
        if (err) {
            console.error('Error fetching students:', err);
            res.status(500).send({ error: 'Something went wrong while fetching students!' });
        } else {
            res.json(results);
        }
    });
});

// Endpoint for uploading files to S3
app.post('/upload', upload.single('file'), (req, res) => {
    const params = {
        Bucket: 'my-learning-platform-bucket', // Replace with your S3 bucket name
        Key: req.file.originalname, // File name
        Body: req.file.buffer, // File content
        ContentType: req.file.mimetype // File type
    };

    s3.upload(params, (err, data) => {
        if (err) {
            console.error('Error uploading file:', err);
            res.status(500).send({ error: 'Failed to upload file.' });
        } else {
            res.send({ message: 'File uploaded successfully!', fileUrl: data.Location });
        }
    });
});
```

```
// Endpoint to list all files in S3 bucket
app.get('/files', (req, res) => {
    const params = {
        Bucket: 'my-learning-platform-bucket',
    };

    s3.listObjectsV2(params, (err, data) => {
        if (err) {
            console.error('Error listing files:', err);
            res.status(500).send({ error: 'Failed to retrieve files.' });
        } else {
            const files = data.Contents.map(file => ({
                name: file.Key,
                lastModified: file.LastModified,
                size: file.Size,
                url: `https://${params.Bucket}.s3.amazonaws.com/${file.Key}`,
            }));
            res.json(files);
        }
    });
});

// Endpoint to download a file from S3 bucket
app.get('/download/:filename', (req, res) => {
    const params = {
        Bucket: 'my-learning-platform-bucket',
        Key: req.params.filename
    };

    s3.getObject(params, (err, data) => {
        if (err) {
            console.error('Error downloading file:', err);
            res.status(500).send({ error: 'Failed to download file.' });
        } else {
            res.setHeader('Content-Type', data.ContentType);
            res.send(data.Body);
        }
    });
});

// Start the server
app.listen(port, '0.0.0.0', () => {
    console.log(`Server is running on port ${port}`);
});
```

## -Testing :-

-first we need to Transfer the file from loacal system to ec2:-

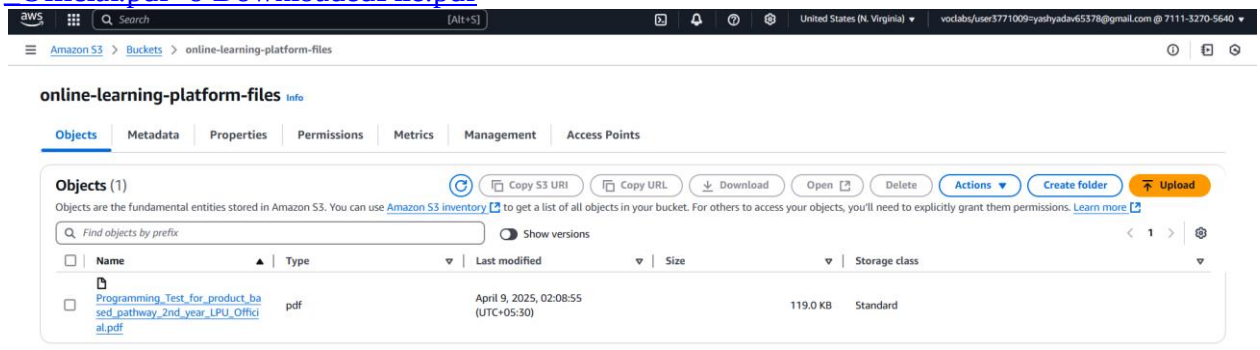
```
scp -i keyone.pem
"C:/Users/kuamr/Downloads/Programming_Test_for_product_based_pathway_2nd_year_LPU_Offi
cial.pdf" ec2-user@ec2-3-88-66-131.compute-1.amazonaws.com:~
```

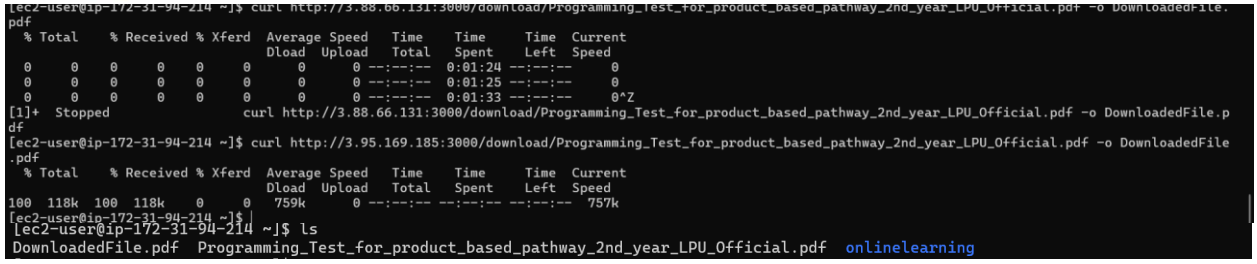
upload file in s3:-

curl -X POST -F "file=@Programming_Test_for_product_based_pathway_2nd_year_LPU_Official.pdf" http://3.88.66.131:3000/upload

download file in ec2:-

curl http://3.88.66.131:3000/download/Programming_Test_for_product_based_pathway_2nd_year_LPU_Official.pdf -o DownloadedFile.pdf




11. create lambda function :-

After that put function:-

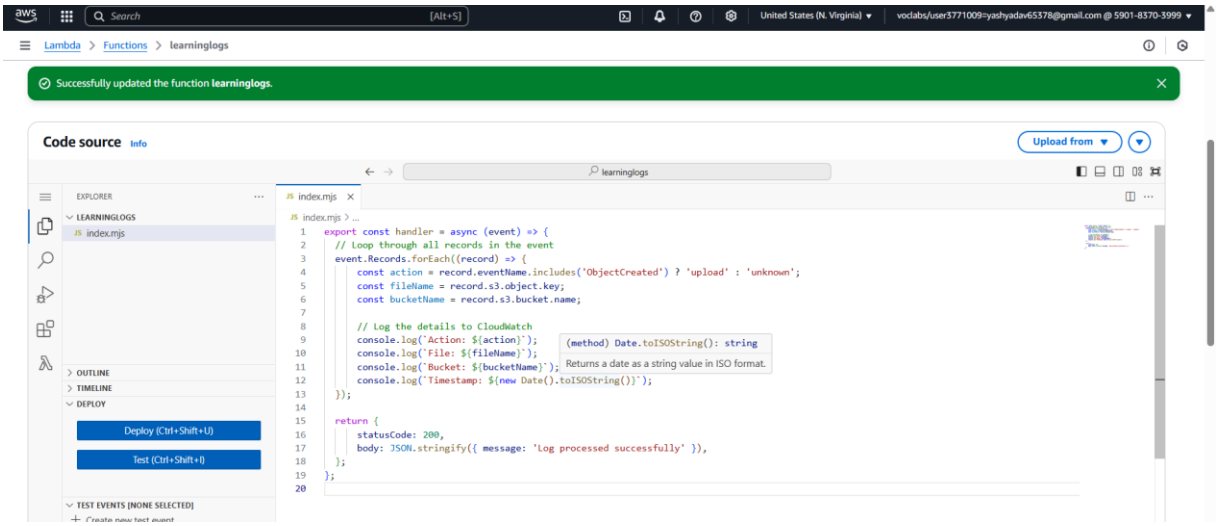
```
export const handler = async (event) => {
    // Loop through all records in the event
    event.Records.forEach((record) => {
        const action = record.eventName.includes('ObjectCreated') ? 'upload' : 'unknown';
        const fileName = record.s3.object.key;
        const bucketName = record.s3.bucket.name;

        // Log the details to CloudWatch
        console.log(`Action: ${action}`);
        console.log(`File: ${fileName}`);
        console.log(`Bucket: ${bucketName}`);
        console.log(`Timestamp: ${new Date().toISOString()}`);
    });

    return {
        statusCode: 200,
        body: JSON.stringify({ message: 'Log processed successfully' }),
    };
};
```

To test things out:-

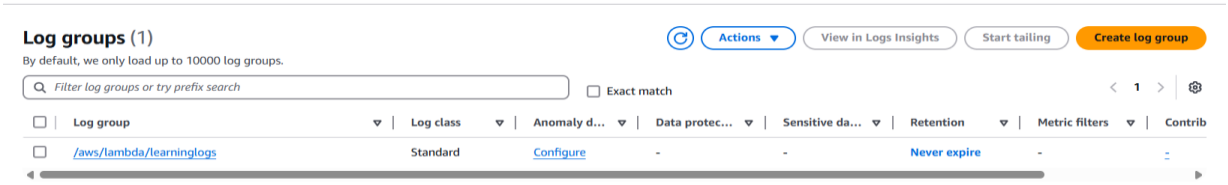
i. Then go to s3 bucket-> properties -> event notifications(configure every thing like put, post, delete etc.)

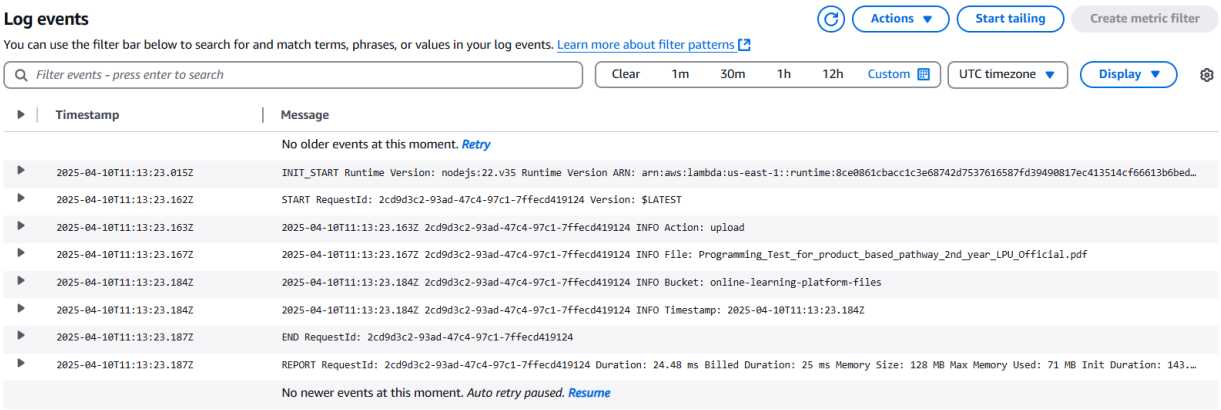
ii. After that try to upload some thing in s3 through cli

iii. And then check logs in logs group-> log events

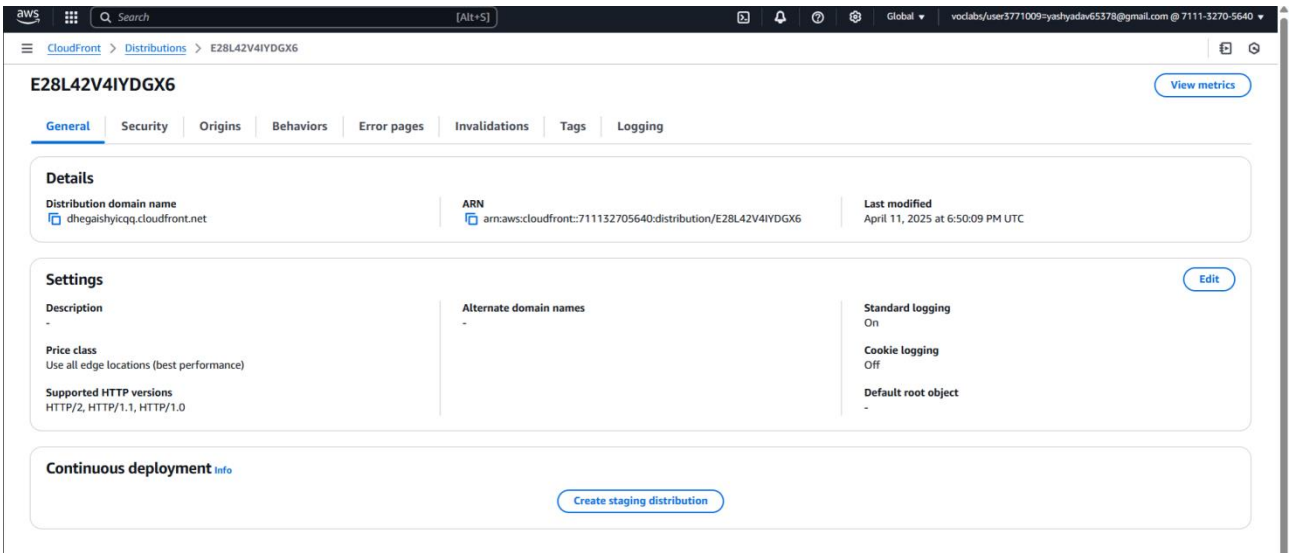



12.come cloud watch and check logs:-




13.Creating Cloud Front:-




-After all the configuration

-Change the S3 Bucket police:-

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Effect": "Allow",
         "Principal": {
            "Service": "cloudfront.amazonaws.com"
         },
         "Action": "s3:GetObject",
         "Resource": "arn:aws:s3:::online-learning-platform-files/*",
         "Condition": {
            "StringEquals": {
               "AWS:SourceArn":
"arn:aws:cloudfront::123456789012:distribution/EDFDVBD632BHDS5"
            }
         }
      }
   ]
}
```

-And disable public access in s3 .

-Also handle core problem so we can connect our backend and frontend which are on ec2 and in my local system:-

-npm install cors

-Change the backend code in index.js(backedfile):-

```
const express = require('express');
const mysql = require('mysql2');
const AWS = require('aws-sdk');
const multer = require('multer');
const cors = require('cors');

const app = express();
const port = 3000;
```

```
// Enable CORS for specific origins
app.use(cors({
    origin: 'http://127.0.0.1:3000', // Allow requests only from your frontend
}));

// Set additional CORS headers (optional)
app.use((req, res, next) => {
    res.setHeader('Access-Control-Allow-Origin', 'http://127.0.0.1:3000'); // Allow frontend origin
    res.setHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS');
    res.setHeader('Access-Control-Allow-Headers', 'Content-Type');
    next();
});

// AWS S3 Configuration
AWS.config.update({
    region: 'us-east-1', // Replace with your bucket region
});
const s3 = new AWS.S3();

// Multer configuration for file uploads
const upload = multer({ storage: multer.memoryStorage() });

// MySQL Database Configuration
const connection = mysql.createConnection({
    host: 'onlinelearningdb.chyic2qaa8mv.us-east-1.rds.amazonaws.com', // Replace with your RDS
endpoint
    user: 'admin', // Replace with your DB username
    password: 'Yash990yash', // Replace with your DB password
    database: 'onlinelearning', // Replace with your database name
});

// Connect to MySQL database
connection.connect((err) => {
    if (err) {
        console.error('Error connecting to the database:', err);
        process.exit(1); // Exit if connection fails
    } else {
        console.log('Connected to the MySQL database!');
    }
});

// Root Endpoint
app.get('/', (req, res) => {
    res.send('Welcome to the Online Learning Platform API!');
});

// Endpoint to Get Courses
app.get('/courses', (req, res) => {
    const instructor = req.query.instructor;
    let query = 'SELECT * FROM courses';
    const queryParams = [];
```

```
    if (instructor) {
        query += ' WHERE instructor = ?';
        queryParams.push(instructor);
    }

    connection.query(query, queryParams, (err, results) => {
        if (err) {
            console.error('Error fetching courses:', err);
            res.status(500).send({ error: 'Something went wrong while fetching courses!' });
        } else {
            res.json(results);
        }
    });
});

// Endpoint to Get Students
app.get('/students', (req, res) => {
    connection.query('SELECT * FROM students', (err, results) => {
        if (err) {
            console.error('Error fetching students:', err);
            res.status(500).send({ error: 'Something went wrong while fetching students!' });
        } else {
            res.json(results);
        }
    });
});

// File Upload Endpoint
app.post('/upload', upload.single('file'), (req, res) => {
    const params = {
        Bucket: 'online-learning-platform-files', // Your S3 bucket name
        Key: req.file.originalname, // Use original file name
        Body: req.file.buffer, // File content
        ContentType: req.file.mimetype, // File type
    };

    s3.upload(params, (err, data) => {
        if (err) {
            console.error('Error uploading file:', err);
            res.status(500).send({ error: 'Failed to upload file.' });
        } else {
            res.send({ message: 'File uploaded successfully!', fileUrl: data.Location });
        }
    });
});

// List Files in S3 Bucket Endpoint
app.get('/files', (req, res) => {
    const params = {
        Bucket: 'online-learning-platform-files', // Your S3 bucket name
    };
```

```
    s3.listObjectsV2(params, (err, data) => {
        if (err) {
            console.error('Error listing files:', err);
            res.status(500).send({ error: 'Failed to retrieve files.' });
        } else {
            const cloudFrontDomain = 'https://d43onenwbtzb1.cloudfront.net'; // Replace with your
CloudFront domain

            const files = data.Contents.map(file => ({
                name: file.Key,
                lastModified: file.LastModified,
                size: file.Size,
                url: `${cloudFrontDomain}/${file.Key}`, // Use CloudFront URL
            }));
            res.json({ files }); // Respond with file list
        }
    });
});

// Download File Endpoint
app.get('/download/:filename', (req, res) => {
    const params = {
        Bucket: 'online-learning-platform-files', // Your S3 bucket name
        Key: req.params.filename, // File name from URL parameter
    };

    s3.getObject(params, (err, data) => {
        if (err) {
            console.error('Error downloading file:', err);
            res.status(500).send({ error: 'Failed to download file.' });
        } else {
            res.setHeader('Content-Type', data.ContentType);
            res.setHeader('Content-Disposition', `attachment; filename="${req.params.filename}"`);
            res.send(data.Body); // Send file data
        }
    });
});

// Start the server
app.listen(port, '0.0.0.0', () => {
    console.log(`Server is running on port ${port}`);
});
```

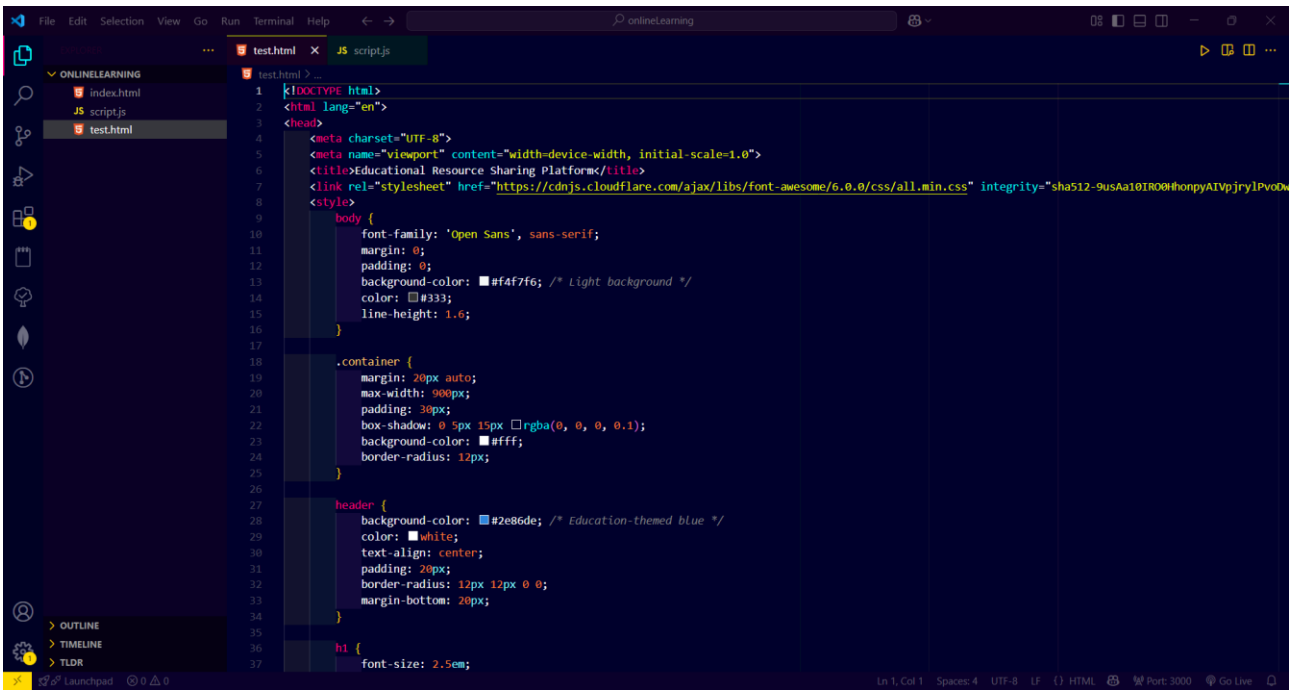
14.In Ec2 our backend code looks like this:-

```
GNU nano 8.3                                index.js                                    Modified
const express = require('express');
const mysql = require('mysql2');
const AWS = require('aws-sdk');
const multer = require('multer');
const cors = require('cors');

const app = express();
const port = 3000;

// Enable CORS for specific origins
app.use(cors({
    origin: 'http://127.0.0.1:3000', // Allow requests only from your frontend
}));

// Set additional CORS headers (optional)
app.use((req, res, next) => {
    res.setHeader('Access-Control-Allow-Origin', 'http://127.0.0.1:3000'); // Allow frontend origin
    res.setHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS');
    res.setHeader('Access-Control-Allow-Headers', 'Content-Type');
    next();
});

// AWS S3 Configuration
AWS.config.update({
    region: 'us-east-1', // Replace with your bucket region
});
const s3 = new AWS.S3();

// Multer configuration for file uploads
const upload = multer({ storage: multer.memoryStorage() });

// MySQL Database Configuration
const connection = mysql.createConnection({
    host: 'onlinelearningdb.chyic2qaa8mv.us-east-1.rds.amazonaws.com', // Replace with your RDS endpoint
    user: 'admin', // Replace with your DB username
    password: 'Yash990yash', // Replace with your DB password
    database: 'onlinelearning', // Replace with your database name
```

15.Our frontend code :-

*frontend is on my local system and backend is hosted on EC2 AWS:-

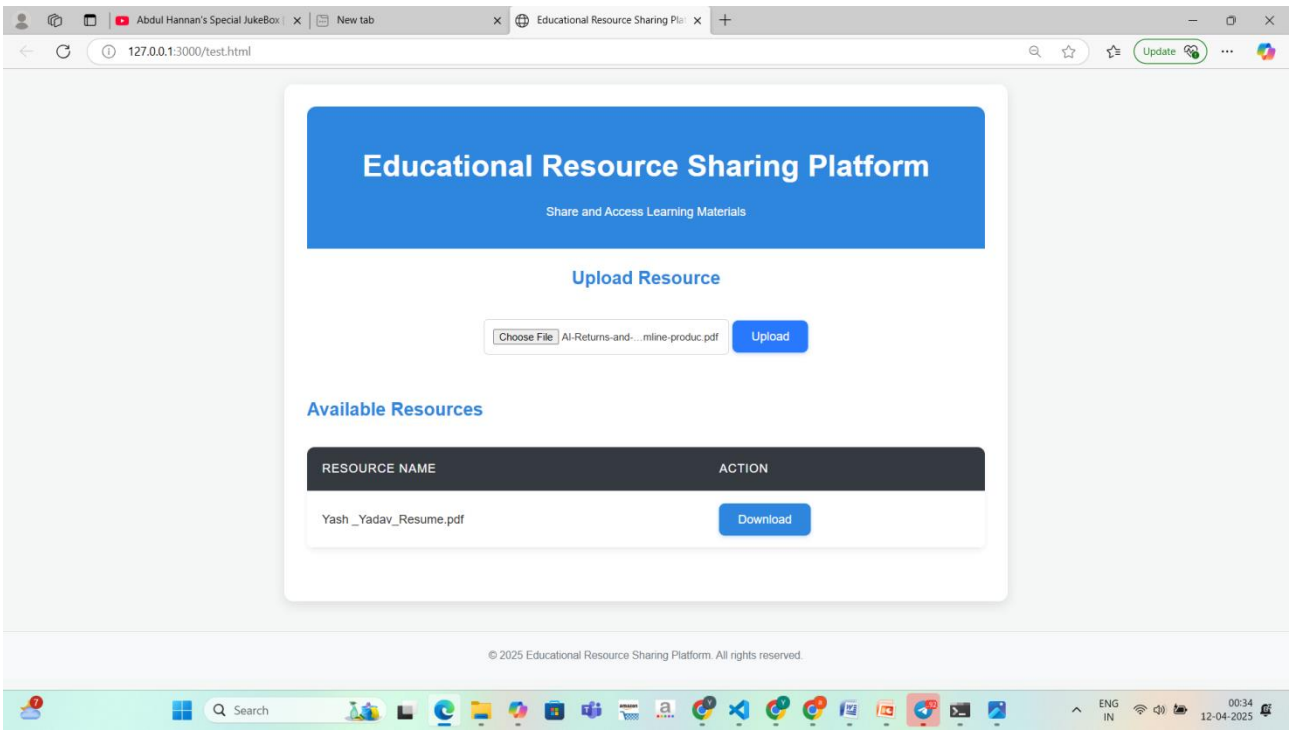
```
// Backend URL
const backendUrl = "http://54.234.175.217:3000"; // Replace with your backend URL

// Get DOM elements
const fileInput = document.getElementById("fileInput");
const uploadButton = document.getElementById("uploadButton");
const fileTableBody = document.getElementById("fileTableBody");

// Function to fetch the file list
const fetchFiles = async () => {
    try {
        const response = await fetch(`${backendUrl}/files`);
        const data = await response.json();
        console.log(data); // Debug: Log backend response in console
        fileTableBody.innerHTML = ""; // Clear existing rows

        // Check if there are any files
        if (!data.files || data.files.length === 0) {
            const row = document.createElement("tr");
            const noFilesCell = document.createElement("td");
            noFilesCell.textContent = "No files found.";
            noFilesCell.setAttribute("colspan", "2");
            row.appendChild(noFilesCell);
            fileTableBody.appendChild(row);
            return;
        }

        // Populate table with files
        data.files.forEach(file => {
            const row = document.createElement("tr");

            const fileNameCell = document.createElement("td");
            fileNameCell.textContent = file.name;

            const actionCell = document.createElement("td");
            const downloadButton = document.createElement("button");
            downloadButton.textContent = "Download";
```

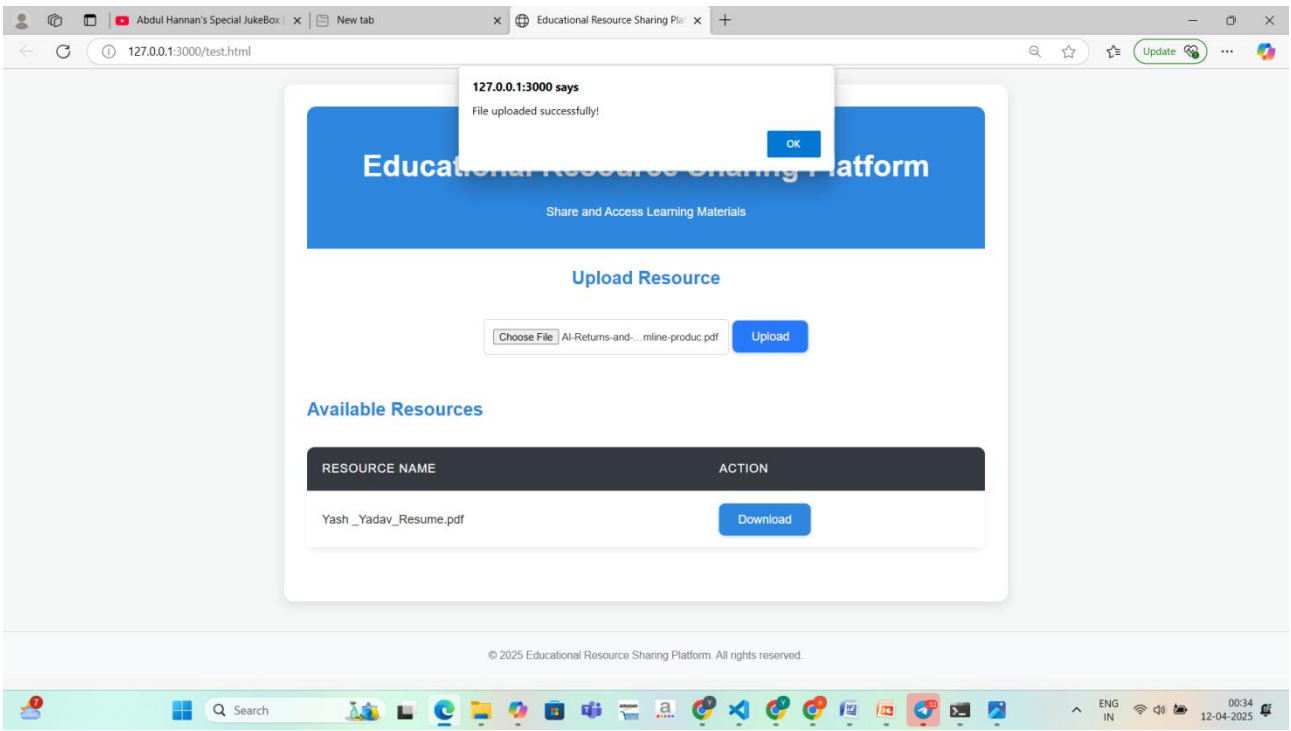



```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Educational Resource Sharing Platform</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css" integrity="sha512-9usAa10IRO0HhonpyAIVpjrylPvoDw
    <style>
        body {
            font-family: 'Open Sans', sans-serif;
            margin: 0;
            padding: 0;
            background-color: #f4f7f6; /* Light background */
            color: #333;
            line-height: 1.6;
        }

        .container {
            margin: 20px auto;
            max-width: 900px;
            padding: 30px;
            box-shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
            background-color: #fff;
            border-radius: 12px;
        }

        header {
            background-color: #2e86de; /* Education-themed blue */
            color: white;
            text-align: center;
            padding: 20px;
            border-radius: 12px 12px 0 0;
            margin-bottom: 20px;
        }

        h1 {
            font-size: 2.5em;
```

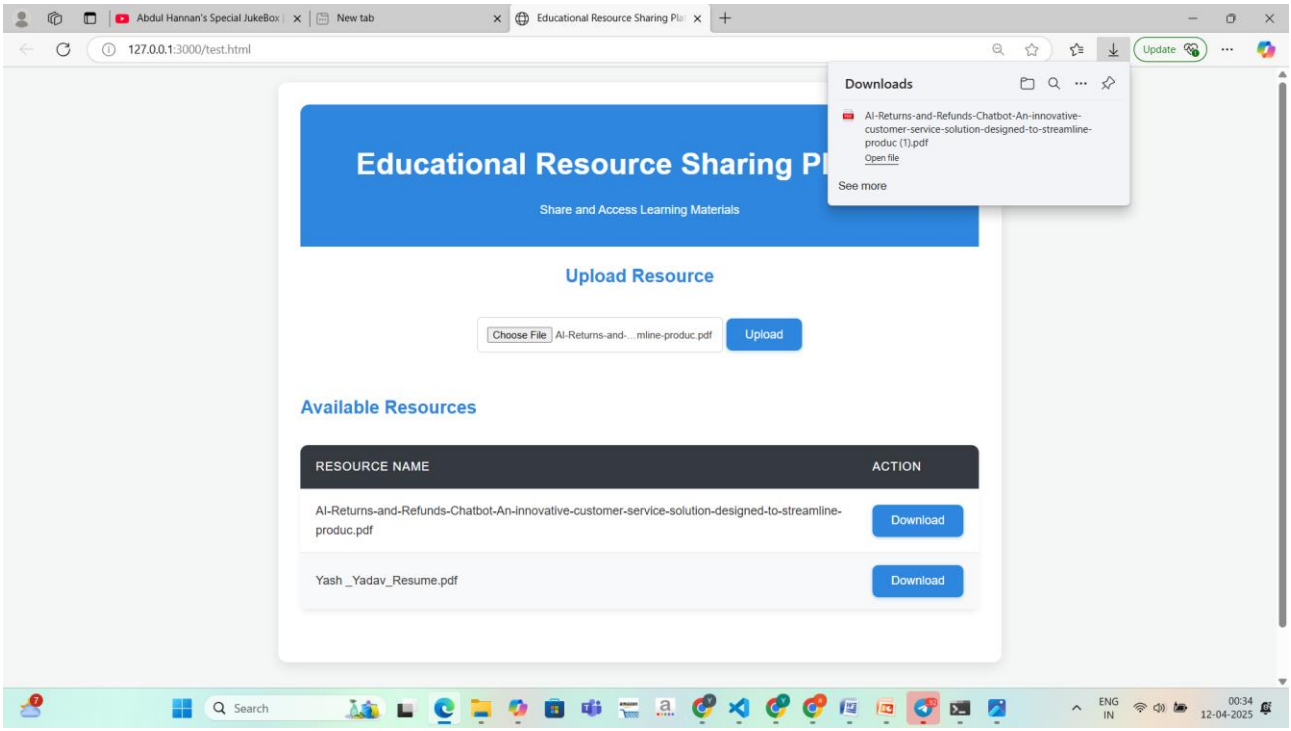
Now after connection our backend and frontend we can try to upload some files and check:-

1.click on upload  and upload file:-






2.can also download file what ever you like to download click on download and download.




<u>Check in s3 bucket and log if files are reached there or not:-</u>

aws | Search [Alt+S] | United States (N. Virginia) | voclabs/user3771009=yashyadav65378@gmail.com @ 7111-3270-5640

Amazon S3 > Buckets > online-learning-platform-files

## online-learning-platform-files Info

Objects | Metadata | Properties | Permissions | Metrics | Management | Access Points

### Objects (2)

Copy S3 URI | Copy URL | Download | Open | Delete | Actions | Create folder | Upload

Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more

Find objects by prefix | Show versions | 1

| Name | Type | Last modified | Size | Storage class |
|---|---|---|---|---|
| AI-Returns-and-Refunds-Chatbot-An-innovative-customer-service-solution-designed-to-streamline-produc.pdf | pdf | April 12, 2025, 00:34:13 (UTC+05:30) | 331.4 KB | Standard |
| Yash_Yadav_Resume.pdf | pdf | April 12, 2025, 00:33:35 (UTC+05:30) | 51.7 KB | Standard |

aws | Search [Alt+S] | United States (N. Virginia) | voclabs/user3771009=yashyadav65378@gmail.com @ 7111-3270-5640

CloudWatch > Log groups > /aws/lambda/logforaction > 2025/04/11/[$LATEST]577138efe895425aa3c5ad3d8a799203

CloudWatch

Favorites and recents

Dashboards

AI Operations Preview

Alarms 0 0 0

Logs

Log groups New

Log Anomalies

Live Tail

Logs Insights New

Contributor Insights

Metrics

X-Ray traces New

Events

Application Signals

Network Monitoring New

Insights

Container Insights

## Log events

Actions | Start tailing | Create metric filter

You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns

Filter events - press enter to search | Clear | 1m | 30m | 1h | 12h | Custom | UTC timezone | Display

| Timestamp | Message |
|---|---|
| | There are older events to load. Load more. |
| 2025-04-11T19:03:35.577Z | INIT_START Runtime Version: nodejs:22.v35 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:8ce0861cbacc1c3e68742d7537616587fd39490817ec413514cf66613b6b… |
| 2025-04-11T19:03:35.724Z | START RequestId: d4da31fe-4348-40b7-98d3-cbfd54d7cbe9 Version: $LATEST |
| 2025-04-11T19:03:35.726Z | 2025-04-11T19:03:35.726Z d4da31fe-4348-40b7-98d3-cbfd54d7cbe9 INFO Action: upload |
| 2025-04-11T19:03:35.738Z | 2025-04-11T19:03:35.738Z d4da31fe-4348-40b7-98d3-cbfd54d7cbe9 INFO File: Yash+_Yadav_Resume.pdf |
| 2025-04-11T19:03:35.739Z | 2025-04-11T19:03:35.739Z d4da31fe-4348-40b7-98d3-cbfd54d7cbe9 INFO Bucket: online-learning-platform-files |
| 2025-04-11T19:03:35.739Z | 2025-04-11T19:03:35.739Z d4da31fe-4348-40b7-98d3-cbfd54d7cbe9 INFO Timestamp: 2025-04-11T19:03:35.739Z |
| 2025-04-11T19:03:35.758Z | END RequestId: d4da31fe-4348-40b7-98d3-cbfd54d7cbe9 |
| 2025-04-11T19:03:35.758Z | REPORT RequestId: d4da31fe-4348-40b7-98d3-cbfd54d7cbe9 Duration: 33.08 ms Billed Duration: 34 ms Memory Size: 128 MB Max Memory Used: 71 MB Init Duration: 14… |
| 2025-04-11T19:04:12.717Z | 2025-04-11T19:04:12.717Z 5565c7f0-a5dc-4485-bf96-821285a8b0f5 INFO Action: upload |
| 2025-04-11T19:04:12.717Z | 2025-04-11T19:04:12.717Z 5565c7f0-a5dc-4485-bf96-821285a8b0f5 INFO File: AI-Returns-and-Refunds-Chatbot-An-innovative-customer-service-solution-designed-to-s… |
| 2025-04-11T19:04:12.717Z | 2025-04-11T19:04:12.717Z 5565c7f0-a5dc-4485-bf96-821285a8b0f5 INFO Bucket: online-learning-platform-files |
| 2025-04-11T19:04:12.717Z | 2025-04-11T19:04:12.717Z 5565c7f0-a5dc-4485-bf96-821285a8b0f5 INFO Timestamp: 2025-04-11T19:04:12.717Z |
| 2025-04-11T19:04:12.717Z | START RequestId: 5565c7f0-a5dc-4485-bf96-821285a8b0f5 Version: $LATEST |
| 2025-04-11T19:04:12.736Z | END RequestId: 5565c7f0-a5dc-4485-bf96-821285a8b0f5 |